

JS niveau avancé : asynchrone et Express

Dompter l'asynchrone et structurer une API Express

Module 1/7

Objectifs du module

Durée : 3h30

- Dompter le paradigme asynchrone : avantages, pièges à éviter, code synchrone, basé sur les callbacks et les promesses
- Maîtriser les promesses et `async / await`, et entrevoir le futur avec ES-Next (ES6+)
- Structurer une API Express : routage, guard (middleware), sécurisation via des stratégies d'authentification (JWT, sessions, passport)

À l'issue du module, vous saurez écrire du code asynchrone robuste et poser les fondations d'une API Express maintenable.

Plan

1. Dompter le paradigme asynchrone : avantages et pièges ; code synchrone, callbacks et promesses
2. Promesses et `async / await` ; le futur avec ES-Next (top-level await, optional chaining, `AbortController`, `structuredClone`)
3. Framework Express : routage, guard (middleware), sécurisation (JWT, sessions, passport)

1. Dompter le paradigme asynchrone

Avantages, pièges, callbacks et promesses

Pourquoi l'asynchrone en Node.js

Node.js est mono-thread pour le JavaScript, mais s'appuie sur une `event loop` et un pool de threads (libuv) pour les I/O.

- Un seul thread JS traite les requêtes, sans blocage pendant les I/O
- Les opérations longues (réseau, disque, base de données) sont déléguées
- Forte densité de connexions concurrentes avec peu de ressources

Le piège : tout code synchrone lourd bloque l'event loop et gèle toutes les requêtes.

Synchrone vs asynchrone

Bloquant (à éviter dans un serveur) :

```
const fs = require("node:fs");

const data = fs.readFileSync(
  "config.json",
  "utf8"
);
console.log(data);
console.log("après");
```

Non bloquant :

```
const fs = require("node:fs");

fs.readFile(
  "config.json",
  "utf8",
  (err, data) => {
    if (err) return console.error(err);
    console.log(data);
  }
);
console.log("après");
```

La version asynchrone affiche **après** avant le contenu du fichier.

Le callback hell

Imbriquer les callbacks rend le code illisible et la gestion d'erreur répétitive.

```
readFile("in.txt", (err, data) => {  
  if (err) return handle(err);  
  callApi(data, (err, res) => {  
    if (err) return handle(err);  
    writeFile("out.txt", res, (err) => {  
      if (err) return handle(err);  
      console.log("terminé");  
    });  
  });  
});
```

Indentation croissante, erreurs dispersées, pas de composition simple.

La convention error-first

En Node.js, le callback reçoit l'erreur en premier argument.

```
function lireConfig(chemin, callback) {  
  fs.readFile(chemin, "utf8", (err, contenu) => {  
    if (err) return callback(err);  
    try {  
      callback(null, JSON.parse(contenu));  
    } catch (e) {  
      callback(e);  
    }  
  });  
}
```

Toujours tester `err` en premier et retourner immédiatement en cas d'erreur.

2. Promesses, async/await et ES-Next

Composer l'asynchrone proprement

Les promesses

Une `Promise` représente une valeur future : `pending`, `fulfilled` ou `rejected`.

```
const lireConfig = (chemin) =>
  new Promise((resolve, reject) => {
    fs.readFile(chemin, "utf8", (err, contenu) => {
      if (err) return reject(err);
      resolve(contenu);
    });
  });

lireConfig("config.json")
  .then((c) => JSON.parse(c))
  .then((cfg) => console.log(cfg))
  .catch((err) => console.error(err));
```

`util.promisify` ou `node:fs/promises` évitent d'écrire ce wrapper à la main.

async / await

`async / await` donne une écriture séquentielle lisible, avec `try / catch` natif.

```
const fs = require("node:fs/promises");

async function traiter(entree, sortie) {
  try {
    const contenu = await fs.readFile(entree, "utf8");
    const res = await appelApi(contenu);
    await fs.writeFile(sortie, res);
    return "terminé";
  } catch (err) {
    console.error("Échec du traitement", err);
    throw err;
  }
}
```

Une fonction `async` retourne toujours une promesse.

Série vs parallèle

En série (lent) :

```
// 3 attentes cumulées
const a = await tache1();
const b = await tache2();
const c = await tache3();
```

En parallèle (rapide) :

```
// lancées simultanément
const [a, b, c] = await Promise.all([
  tache1(),
  tache2(),
  tache3(),
]);
```

Utiliser `Promise.all` quand les tâches sont indépendantes. `Promise.allSettled` quand on veut tous les résultats malgré les échecs.

Pièges classiques de l'asynchrone

- `await` en série dans une boucle alors que les itérations sont indépendantes
- Oublier `await` (ou `return`) : promesse non attendue, erreur silencieuse
- `unhandledRejection` : une promesse rejetée sans `catch` peut arrêter le process
- Mélanger callbacks et promesses sur une même fonction
- Capturer trop large : `try / catch` qui masque la vraie origine de l'erreur

Le futur avec ES-Next (ES6+)

- `top-level await` : `await` au niveau module (ES modules) sans fonction englobante
- `optional chaining` : `user?.profile?.email` sans cascade de tests
- `AbortController` : annuler un `fetch` ou un timer proprement
- `structuredClone` : copie profonde native, sans `JSON.parse(JSON.stringify(...))`

```
const controller = new AbortController();
setTimeout(() => controller.abort(), 2000);
const res = await fetch(url, { signal: controller.signal });
```

3. Framework Express

Routage, guard et sécurisation

Routage Express

Express associe une méthode HTTP et un chemin à un gestionnaire.

```
const express = require("express");
const app = express();

app.use(express.json());

app.get("/api/users/:id", (req, res) => {
  res.json({ id: req.params.id });
});

app.listen(3000, () => console.log("port 3000"));
```

Les **Router** permettent de découper les routes par domaine fonctionnel.

Architecture en couches

Séparer les responsabilités pour la maintenabilité et les tests.

routes/	définissent les endpoints et branchent les middlewares
controllers/	lisent la requête, appellent les services, renvoient la réponse
services/	logique métier, indépendante d'Express
middlewares/	transverses : erreurs, authentication, validation

Le controller ne contient pas de logique métier ; le service ne connaît pas `req / res` .

Guard : le middleware

Un middleware reçoit `(req, res, next)` et décide de poursuivre ou non.

```
function guardAuth(req, res, next) {  
  const header = req.headers.authorization;  
  if (!header?.startsWith("Bearer ")) {  
    return res.status(401).json({ error: "Token manquant" });  
  }  
  // validation du token...  
  next();  
}  
  
app.get("/api/prive", guardAuth, controller.lister);
```

`next(err)` transfère l'erreur au middleware d'erreur centralisé.

Middleware d'erreur centralisé

Un middleware à 4 arguments capture toutes les erreurs des routes.

```
function gestionErreurs(err, req, res, next) {  
  const status = err.status || 500;  
  res.status(status).json({  
    error: err.message || "Erreur serveur",  
  });  
}  
  
// déclaré en dernier, après toutes les routes  
app.use(gestionErreurs);
```

Dans les handlers `async`, propager l'erreur via `next(err)` ou un wrapper.

Stratégies d'authentification

JWT (stateless) :

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
  { sub: user.id },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);
const payload = jwt.verify(
  token,
  process.env.JWT_SECRET
);
```

Sessions / passport :

- Sessions : état serveur, cookie d'identifiant
- `passport` : couche d'abstraction multi-stratégies (local, JWT, OAuth)
- JWT : pas d'état serveur, idéal API et microservices

Choisir selon le besoin : API publique stateless (JWT) ou application avec session serveur.

Démo

Reproduire et corriger les pièges classiques de l'asynchrone : `unhandledRejection`, `await` en série au lieu de `Promise.all`, callback hell.

Voir `demos/demo-01.md`

Travaux pratiques

Construire une API Express structurée : couches routes/controllers/services, middleware d'erreur centralisé, guard d'authentification JWT et validation.

Durée : environ 1h30

Fichier : `tp/tp-01.md`

Récapitulatif

- L'event loop rend Node.js performant en I/O ; ne jamais bloquer le thread principal
- Préférer promesses et `async / await` aux callbacks imbriqués
- Paralléliser les tâches indépendantes avec `Promise.all`
- ES-Next simplifie : top-level await, optional chaining, `AbortController`, `structuredClone`
- Structurer Express en couches avec guard et middleware d'erreur centralisé

Cheatsheet : `cheatsheet/module-01.md`

Questions

Module suivant : Streams Node.js